



Universidade do Minho

Licenciatura em Engenharia Informática - LEI

Computação Gráfica

Trabalho Prático - Fase 2

Grupo 14

A107324 - David José Barbosa Alves

A107368 - Diogo Malheiro Pais

A107377 - João Pedro Ferreira Gomes

A107332 - Pedro Afonso Quesado Pacheco Pereira

1. Introdução

Esta fase representa a conclusão do motor gráfico, completando o pipeline de renderização com iluminação e texturização. Partindo de toda a infraestrutura construída nas fases anteriores — geração de primitivas (Fase 1), Scene Graph hierárquico (Fase 2), VBOs e animações com Catmull-Rom (Fase 3) — as alterações desta fase foram deliberadamente cirúrgicas: o que já funcionava não foi tocado.

As duas componentes com trabalho novo são:

- **Generator:** Generator: o formato de saída .3d foi expandido para incluir normais e coordenadas de textura (UV) por vértice. Os algoritmos de geração das primitivas já existentes foram atualizados; uma nova primitiva (anel) foi adicionada.
- **Engine:** Engine: passou a carregar três VBOs por modelo (vértices, normais, UV), a gerir texturas via DevIL, a aplicar materiais por modelo, e a configurar fontes de luz lidas do XML.

O Generator não tinha alterações nas fases 2 e 3. Na Fase 4 volta a ter trabalho: cada primitiva passou a calcular e exportar normais e coordenadas UV, o que obrigou a alterar a estrutura de dados interna e o formato do ficheiro .3d.

2. Expansão do Generator: Normais e Coordenadas UV

A estrutura Vertex em utils.h foi atualizada para conter os oito campos necessários ao pipeline de iluminação e texturização:

```
struct Vertex {  
    float x, y, z;      // posição  
    float nx, ny, nz;   // normal  
    float u, v;         // coordenadas de textura  
};
```

O ficheiro .3d manteve a linha de cabeçalho com o número de vértices (usada pela alocação dinâmica do Engine desde a Fase 1), mas cada linha de vértice passou de 3 para 8 valores: x y z nx ny nz u v. Esta é a única alteração ao protocolo entre Generator e Engine nesta fase.

2.1 Esfera

A esfera é o caso mais simples: sendo uma superfície equidistante do centro, a normal em cada ponto é o próprio vetor posição normalizado. Não há cálculo adicional.

```
vtx.nx = vtx.x / raio;  
vtx.ny = vtx.y / raio;  
vtx.nz = vtx.z / raio;
```

As coordenadas UV mapeiam diretamente os ângulos paramétricos α e β para $[0,1] \times [0,1]$, resultando numa projeção equiretangular compatível com as texturas planetárias utilizadas na cena demo.

2.2 Plano

Normal constante (0, 1, 0). As coordenadas UV crescem linearmente com o índice da célula da grelha, cobrindo $[0,1] \times [0,1]$ para qualquer número de divisões.

2.3 Caixa (Box)

Cada face tem uma normal fixa ortogonal à sua direção, e as coordenadas UV cobrem cada face de forma independente. A orientação CCW já garantida desde a Fase 1 assegura que as normais apontam para o exterior sem cálculo adicional.

2.4 Cone

A base usa normal (0, -1, 0). A superfície lateral é o único caso que requer cálculo explícito: a inclinação do cone implica que a normal lateral tem componente Y e componente horizontal determinadas pela relação entre raio e altura:

```
float len      = sqrt(raio*raio + altura*altura);  
float ny_n     = raio / len;      // componente Y  
float horiz    = altura / len;    // componente horizontal  
vtx.nx = sin(angulo) * horiz;  
vtx.ny = ny_n;  
vtx.nz = cos(angulo) * horiz;
```

2.5 Patches de Bézier

As normais para as patches já eram necessárias para a iluminação e foram calculadas analiticamente a partir das derivadas parciais da superfície, usando as derivadas dos polinómios de Bernstein implementados na Fase 3. O produto vetorial dos dois vetores tangentes dá a normal:

```
// tangente dU: derivada de Bernstein em U, polinómio em V  
dU += cp * bernsteinDerivative(i,u) * bernstein(j,v);  
// tangente dV: polinómio em U, derivada de Bernstein em V  
dV += cp * bernstein(i,u) * bernsteinDerivative(j,v);  
// normal = dV × dU  
crossProduct(dV, dU, p.nx, p.ny, p.nz);
```

As coordenadas UV são os próprios parâmetros U e V da superfície, garantindo um mapeamento natural e contínuo.

2.6 Nova Primitiva: Anel (Ring)

O anel de Saturno exigiu uma nova primitiva. É gerado como uma coroa circular com raio interno e externo configuráveis, e renderizado com duas faces (superior e inferior) para ser visível de qualquer ângulo de câmara. As normais das duas faces são opostas:

```
// Face superior: ny = +1
v.push_back({r*sin(a), 0, r*cos(a), 0, +1, 0, u, v_rad});
// Face inferior: ny = -1 (ordem invertida para CCW)
p.ny = -1;
v.push_back(p3); v.push_back(p2); v.push_back(p1);
```

A transparência do anel é tratada no Engine com GL_BLEND, ativado na inicialização. O comando de invocação do Generator para o anel é: `.generator ring <raio_int> <raio_ext> <slices> <ficheiro.3d>`

[INSERIR: Primitivas geradas com iluminação ativa, mostrando o efeito das normais]

Figura 1: Efeito das normais nas primitivas com iluminação ativa.

3. Engine: Três VBOs por Modelo

Na Fase 3, cada modelo usava um único VBO para os vértices. Nesta fase, o `loadModel` passa a criar três VBOs distintos e a popular três vetores separados durante a leitura do ficheiro `.3d`:

```
glGenBuffers(1, &m.vboVertices); // posições x y z
glGenBuffers(1, &m.vboNormals); // normais nx ny nz
glGenBuffers(1, &m.vboTexCoords); // coordenadas u v
```

Durante a renderização, os três client state arrays são ativados antes do loop de grupos e desativados depois, e cada VBO é ligado ao ponteiro correto imediatamente antes do `drawcall`:

```
glEnableClientState(GL_VERTEX_ARRAY);
glEnableClientState(GL_NORMAL_ARRAY);
glEnableClientState(GL_TEXTURE_COORD_ARRAY);

glBindBuffer(GL_ARRAY_BUFFER, model.vboVertices); glVertexPointer(3,
GL_FLOAT, 0, 0);
```

```
glBindBuffer(GL_ARRAY_BUFFER, model.vboNormals);    glNormalPointer(GL_FLOAT,
0, 0);
glBindBuffer(GL_ARRAY_BUFFER, model.vboTexCoords); glTexCoordPointer(2,
GL_FLOAT, 0, 0);
glDrawArrays(GL_TRIANGLES, 0, model.vertexCount);
```

Esta alteração é a única mudança ao pipeline de renderização herdado da Fase 3. A estrutura de grupos, a Matrix Stack e as transformações animadas não foram alteradas.

4. Materiais

Cada modelo passou a ter uma estrutura Material associada, com valores padrão neutros que são substituídos se o XML definir uma tag <color>:

```
struct Material {
    float diffuse[4]  = {0.8f, 0.8f, 0.8f, 1.0f};
    float ambient[4]  = {0.2f, 0.2f, 0.2f, 1.0f};
    float specular[4] = {0.0f, 0.0f, 0.0f, 1.0f};
    float emissive[4] = {0.0f, 0.0f, 0.0f, 1.0f};
    float shininess   = 0.0f;
};
```

A função parseColor lê os valores R, G, B de cada componente do XML e converte-os para o intervalo [0,1] antes de os guardar no material. Antes de cada drawcall, o motor aplica o material:

```
glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE,    model.material.diffuse);
glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT,    model.material.ambient);
glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR,   model.material.specular);
glMaterialfv(GL_FRONT_AND_BACK, GL_EMISSION,   model.material.emissive);
glMaterialf (GL_FRONT_AND_BACK, GL_SHININESS, model.material.shininess);
```

A componente emissiva é particularmente relevante na cena demo: o Sol e o fundo de estrelas têm emissive = (255, 255, 255), tornando-os completamente auto-iluminados e independentes das fontes de luz. Esta é a forma correta de representar corpos que emitem luz própria sem afetarem a iluminação dos outros.

Exemplo de configuração XML de material:

```
<model file="sphere.3d">
  <color>
    <diffuse  R="200" G="200" B="200" />
    <ambient  R="50"  G="50"  B="50"  />
    <specular  R="100" G="100" B="100" />
    <emissive  R="0"   G="0"   B="0"   />
    <shininess value="32" />
  </color>
</model>
```

5. Texturização

O carregamento de texturas foi implementado com a biblioteca DevIL (Developer's Image Library), que permite carregar JPEG, PNG e outros formatos diretamente para texturas OpenGL. A função `loadTexture` trata de todo o processo:

```
ilEnable(IL_ORIGIN_SET);
ilOriginFunc(IL_ORIGIN_LOWER_LEFT); // corrige eixo Y invertido pelo DevIL
ilGenImages(1, &image); ilBindImage(image);
ilLoadImage((ILstring)filename.c_str());
ilConvertImage(IL_RGBA, IL_UNSIGNED_BYTE);
glGenTextures(1, &texID); glBindTexture(GL_TEXTURE_2D, texID);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, w, h, 0, GL_RGBA, GL_UNSIGNED_BYTE,
data);
```

O problema mais comum com DevIL é a orientação invertida das texturas. A solução foi ativar `IL_ORIGIN_SET` com `IL_ORIGIN_LOWER_LEFT`, que instrui o DevIL a inverter o eixo Y durante o carregamento, sem necessidade de ajustar as coordenadas UV geradas pelo Generator.

Cada modelo suporta várias texturas, guardadas num vector de IDs. Na renderização, o motor liga a primeira textura disponível; se não existir nenhuma, desativa `GL_TEXTURE_2D` para esse modelo:

```
if (model.texIDs.size() >= 1 && model.texIDs[0] != 0) {
    glEnable(GL_TEXTURE_2D);
    glBindTexture(GL_TEXTURE_2D, model.texIDs[0]);
} else {
    glDisable(GL_TEXTURE_2D);
}
```

No XML, as texturas são declaradas com tags `<texture>` dentro de `<model>`. A Terra é um caso especial com duas texturas (mapa diurno e noturno):

```
<model file="sphere.3d">
    <texture file="textures/earth_daymap.jpg" />
    <texture file="textures/earth_nightmap.jpg" />
</model>
```

[INserir: Screenshots das cenas de teste 4_1 a 4_6 comparadas com as imagens de referência]

Figura 2: Comparação entre a renderização do motor (esquerda) e as imagens de referência da Fase 4 (direita).

6. Iluminação

O motor suporta até 8 fontes de luz OpenGL (GL_LIGHT0 a GL_LIGHT7), lidas do XML e configuradas no início de cada frame antes do desenho dos grupos. A tag <lights> é um filho direto de <world>, ao mesmo nível de <camera> e <group>.

6.1 Tipos de Luz e Leitura do XML

São suportados três tipos. A distinção entre pontual/spot (posição) e direcional (direção) é feita através do quarto componente w do vetor de posição OpenGL — convenção standard do OpenGL que evita um if extra na renderização:

- Pontual (type="point"): w = 1.0. Irradia em todas as direções a partir de uma posição. Parâmetros: posX, posY, posZ.
- Direcional (type="directional"): w = 0.0. Simula fonte infinitamente distante; a posição é interpretada como direção. Parâmetros: dirX, dirY, dirZ.
- Spotlight (type="spot"): w = 1.0, mais GL_SPOT_DIRECTION e GL_SPOT_CUTOFF. Parâmetros: posX/Y/Z, dirX/Y/Z, cutoff.

```
<lights>
  <light type="point"      posX="0"  posY="0"  posZ="0" />
  <light type="directional" dirX="1"  dirY="1"  dirZ="1" />
  <light type="spot"       posX="0"  posY="10" posZ="0"
                                dirX="0"  dirY="-1" dirZ="0" cutoff="30" />
</lights>
```

6.2 Decisões de Implementação

Três decisões técnicas foram tomadas para garantir resultados corretos, todas elas referenciadas explicitamente nas notas da Fase 4 fornecidas pela equipa docente:

- **GL_RESCALE_NORMAL:** GL_RESCALE_NORMAL: ativado na inicialização para que as normais sejam renormalizadas após transformações de escala. Sem isto, modelos com <scale> apresentam iluminação incorreta.
- **Ambient global:** GL_LIGHT_MODEL_AMBIENT: configurado com (1, 1, 1, 1) para que a componente ambiente dos materiais seja reproduzida independentemente das luzes. Necessário para o Sol e estrelas com emissive total.
- **Modelo de luz:** GL_LIGHT_MODEL_TWO_SIDE + GL_LIGHT_MODEL_LOCAL_VIEWER: ativados para iluminação correta nas faces internas e cálculos de especularidade mais precisos.

As luzes são posicionadas depois de `glLoadIdentity` e antes de qualquer `renderGrupo`, garantindo que as suas coordenadas são interpretadas em espaço de mundo e não de objeto.

[INSERIR: Comparação dos três tipos de luz nas cenas de teste]

Figura 3: Efeito dos três tipos de luz (pontual, direcional, spotlight) nas cenas de teste.

7. Cena Demo: Sistema Solar Texturizado

A cena demo é a evolução direta do sistema solar dinâmico da Fase 3. A hierarquia de grupos, os offsets angulares, as órbitas por Catmull-Rom e as rotações temporizadas foram mantidos sem alteração. O que mudou foi exclusivamente a camada visual: cada corpo passou a ter textura real e material configurado.

Configurações relevantes no `sistema_solar.xml`:

- Fundo estelar: esfera grande com textura `stars_milky_way.jpg` e emissive branco total, não afetada pela iluminação.
- Sol: textura `sun.jpg`, emissive branco total. A fonte de luz pontual está posicionada na origem (0,0,0), coincidente com o Sol.
- Planetas: cada um com a sua textura (`mercury.jpg`, `venus.jpg`, `earth_daymap.jpg`, `mars.jpg`, `jupiter.jpg`, `saturn.jpg`, `uranus.jpg`, `neptune.jpg`).
- Terra: caso especial com duas texturas declaradas (mapa diurno e noturno).
- Anel de Saturno: primitiva ring com textura `saturn_ring.png` semi-transparente via `GL_BLEND`.
- Cometa (Utah Teapot): textura `teapot.jpg`, mantido da Fase 3.

Exemplo da configuração da Terra, que ilustra a combinação de transformações animadas herdadas da Fase 3 com a texturização nova desta fase:

```
<group> <!-- Terra: órbita -->
  <transform>
    <rotate angle="200" x="0" y="1" z="0" /> <!-- offset orbital (Fase
3) -->
    <rotate time="36.5" x="0" y="1" z="0" /> <!-- órbita (Fase 3) -->
    <translate x="52" y="0" z="0" />
  </transform>
  <group> <!-- Terra: rotação própria + textura (Fase 4) -->
    <transform>
      <rotate angle="23.4" x="0" y="0" z="1" /> <!-- inclinação axial
-->
      <rotate time="1" x="0" y="1" z="0" /> <!-- 1 dia -->
    </transform>
  </group>
</models>
```

```
<model file="sphere.3d">
  <texture file="textures/earth_daymap.jpg" />
  <texture file="textures/earth_nightmap.jpg" />
</model>
</models>
</group>
</group>
```

[INserir: Vista panorâmica do Sistema Solar completo com texturas e iluminação]

Figura 4: Vista panorâmica do Sistema Solar com texturização e iluminação completas.

[INserir: Detalhe da Terra (textura diurna/noturna) e Lua em órbita]

Figura 5: Detalhe da Terra com mapa diurno/noturno e a Lua em órbita.

[INserir: Saturno com anel semi-transparente texturizado]

Figura 6: Saturno com anel semi-transparente.

8. Resultados e Testes

As seis cenas de teste fornecidas para a Fase 4 cobrem progressivamente os diferentes aspetos implementados:

- test_4_1 a test_4_3: validação dos três tipos de luz com primitivas simples e materiais estáticos.
- test_4_4 a test_4_5: validação de materiais com diferentes combinações de componentes e shininess.
- test_4_6: validação da texturização nas quatro primitivas base (esfera, caixa, cone, plano).

[INserir: Comparação direta motor (esquerda) vs referência (direita) para test_4_1 a test_4_6]

Figura 7: Validação das seis cenas de teste da Fase 4.

9. Conclusão

A Fase 4 concluiu o motor gráfico. As alterações foram deliberadamente mínimas: expandir o formato .3d, adicionar três VBOs por modelo, integrar DevIL para texturas, e configurar o

pipeline de iluminação OpenGL. Toda a arquitetura herdada das fases anteriores — Scene Graph, Matrix Stack, animações, câmara orbital — funcionou sem modificações.

O resultado é um motor capaz de renderizar cenas complexas e animadas com iluminação e texturas reais. Os próximos passos naturais seriam a substituição do pipeline de função fixa por shaders GLSL e a implementação de view frustum culling para otimização da renderização.

10. Referências Bibliográficas

1. OpenGL Registry: "OpenGL 2.1 Reference — glMaterial, glLight, glLightModel." <https://www.opengl.org/>
2. DevIL (Developer's Image Library): <http://openil.sourceforge.net>
3. LearnOpenGL.com: "Lighting" e "Textures." <https://learnopengl.com>
4. Shreiner, D. et al.: "OpenGL Programming Guide, 8th Edition" (The Red Book).
5. Catmull, E., & Rom, R.: "A class of local interpolating splines." (mantida das fases anteriores)
6. TinyXML-2: <https://github.com/leethomason/tinyxml2> (mantida das fases anteriores)